Substrate Architecture

Patterns stolen from LangGraph, LangChain, and LangSmith. Not using their names or their hosted services. Using their abstractions — the ones that survived production.

The Stack

```
│   tracing, evaluation, feedback          │
└──────────────────────────────────────────────────
──────────────────────────┘
```

Each layer knows its own contracts. None reaches into another.

What We Steal From LangGraph

1. The Port Pattern (their best idea)

LangGraph's checkpoint library is the cleanest example of the port/adapter pattern
in any major open-source AI project. Read this and internalize it:

```
libs/checkpoint/          ← the INTERFACE
(the port)
  checkpoint/base/        ←
BaseCheckpointSaver (abstract)
  checkpoint/serde/base   ←
SerializerProtocol (Protocol class)
  store/base/             ← BaseStore
(abstract)

libs/checkpoint-postgres/ ← one ADAPTER
libs/checkpoint-sqlite/   ← another ADAPTER
libs/checkpoint/memory/   ← nursery-grade
ADAPTER
```

The interface in `serde/base.py` is 20 lines. Two methods: `dumps_typed` and `loads_typed`. Every adapter implements those two methods. App code calls only those two. Swap postgres for sqlite: zero app changes.

Steal this structure for every substrate port:

```
substrate/
  ports/
    <name>/
      __init__.py     ← the interface (Protocol or ABC, ~20 lines)
      nursery.py      ← in-memory / minimal real impl
      <adapter>.py    ← heavier impl added when project needs it
```

2. Typed State Channels

LangGraph's channels (`channels/base.py`) enforce contracts on what flows between nodes. `BaseChannel` is generic over `Value`, `Update`, and `Checkpoint` — the type of what you read, what you write, and what gets persisted are all declared.

What this means for substrate:
When blocks compose, the data flowing between them has a type. Not an arbitrary dict.

If the auth block outputs a `User`, the database block's query scope takes a `User`.
The contract is in the type, not in a doc.

Start simple: use Python `TypedDict` or dataclasses at each port boundary.
Evolve toward Pydantic when you need validation.

```
# The thing that crosses the auth port
boundary
class AuthedUser(TypedDict):
    user_id: str
    email: str
    roles: list[str]
```

3. The `@task` / `@entrypoint` Pattern

LangGraph's functional API (`func/__init__.py`) is the "invisible blocks" pattern
made real. You annotate a function; the framework handles state, retry, caching,
checkpointing. The developer writes the capability; the infrastructure is injected.

```
@task(retry_policy=RetryPolicy(max_attempts=3
))
async def call_external_api(input: str) ->
str:
    ...  # just the capability, no retry
boilerplate
```

What to steal: the decorator as the composition surface. A substrate block is
a decorator that injects the port implementation:

```
@with_auth         # injects auth port
@with_db           # injects db port
@traced            # injects observability
port
async def my_handler(user: AuthedUser, db:
Database) -> Response:
    ...  # just the business logic
```

The ports arrive as injected dependencies, not as imports.
The app never sees the adapter. The decorator is the port.

4. Protocol-Based Interfaces (not ABC inheritance)

LangGraph uses Python `Protocol` classes (`runtime_checkable=True`) not ABC
inheritance for its core interfaces. This is the right call:

```
@runtime_checkable
class SerializerProtocol(Protocol):
    def dumps_typed(self, obj: Any) ->
tuple[str, bytes]: ...
    def loads_typed(self, data: tuple[str,
bytes]) -> Any: ...
```

Any class that has those two methods is a valid serializer. No inheritance required.
Duck typing with runtime checking. Easy to swap, easy to test with fakes.

Use `Protocol` for all substrate port interfaces. Not ABC. This means:

- Any object satisfying the interface works (including 3rd party classes)
- Test fakes are trivial (just implement the methods)
- No import dependency on substrate from the implementation

5. Namespaced Key-Value Store

LangGraph's `BaseStore` (`store/base/__init__.py`) is a hierarchical key-value store
with namespaces as tuples: `("user", "alice", "preferences")`. It supports put, get,
search (with optional vector similarity), and list. Used for long-term memory that
persists across agent runs.

This is the FoT insight store pattern made concrete. The substrate insight store
should adopt this interface:

```
# A port's cross-project insight store
store.put(("substrate", "auth", "insights"),
"oauth-refresh", {
```

```
    "lesson": "OAuth tokens expire; always
handle refresh at the port boundary",
    "source": "project-x",
    "date": "2026-06-08"
})

# Next project pulls it
items = store.search(("substrate", "auth",
"insights"))
```

Nursery impl: in-memory dict. Graduated impl: sqlite or postgres. Same interface.

What We Steal From LangSmith

1. The `@traceable` Decorator

LangSmith's `run_helpers.py` exports `@traceable` — wrap any function, all inputs,
outputs, errors, and timing are automatically captured and sent to the tracing backend.

We want the same pattern but backend-agnostic:

```
@traced(name="auth.login", tags=["auth"])
async def login(user: str, pw: str) ->
AuthedUser:
    ...
```

The tracing port interface:

```
class TracingPort(Protocol):
    def trace(
        self,
        name: str,
        inputs: dict,
        outputs: dict | None = None,
        error: Exception | None = None,
        tags: list[str] | None = None,
    ) -> None: ...
```

Nursery adapter: structured stdout (JSON). Graduated adapter: LangSmith SDK, Honeycomb, Datadog — all behind the same interface.

2. Evaluation as a Port

LangSmith's evaluation framework (`evaluation/_runner.py`) treats scoring as a first-class operation: given inputs and expected outputs, run evaluators, collect scores. This is a port:

```
class EvaluationPort(Protocol):
    def evaluate(
        self,
        inputs: dict,
        outputs: dict,
        reference: dict | None = None,
    ) -> list[EvalResult]: ...
```

Nursery: simple assertion-based evaluator.
Graduated: LLM-as-judge, custom scorers.

The Nursery: How a New Project Starts

A new project clones or copies `substrate/nursery/`.
It gets:

```
my-project/
  substrate/
    ports/
      auth/
        __init__.py         ← AuthPort
Protocol (20 lines)
        nursery.py          ← LocalAuth
(hashed pw + random token)
      db/
        __init__.py         ← DBPort
Protocol
        nursery.py          ← SQLite or in-
memory
      schema/
        __init__.py         ← SchemaPort
Protocol
        nursery.py          ← baseline
tables (id, created_at, updated_at)
      transport/
        __init__.py         ← TransportPort
Protocol
```

```
        nursery.py          ← HTTP via
stdlib or minimal framework
      tracing/
        __init__.py         ← TracingPort
Protocol
        nursery.py          ← structured
JSON to stdout
      config/
        __init__.py         ← ConfigPort
Protocol
        nursery.py          ← env vars via
os.environ, fail-loud on missing
      store/
        __init__.py         ← StorePort
Protocol (stolen from LangGraph)
        nursery.py          ← in-memory dict
    compose.py              ← wires ports
together, injects into app
    insights/               ← FoT: cross-
project lessons per port
      auth.json
      db.json
      transport.json
  app/
    ... your code, imports only from
substrate/ports/*, never from adapters
```

Rules:

1. `app/` imports from `substrate/ports/*/` only. Never from `substrate/ports/*/nursery.py` directly.

2. `compose.py` is the only file that names adapters. It creates instances and injects.
3. `insights/*.json` is append-on-use: when a lesson is learned, it lands here and gets distilled across projects (FoT).

How Ports Compose (Meta-Agent Pattern)

Ports compose via typed boundaries. The compose layer wires them:

```
# substrate/compose.py — the ONLY place that
names adapters
from substrate.ports.auth.nursery import
LocalAuth
from substrate.ports.db.nursery import
SQLiteDB
from substrate.ports.tracing.nursery import
StdoutTracer

def create_container():
    db = SQLiteDB(path=":memory:")
    auth = LocalAuth(db=db)
    tracer = StdoutTracer()
    return Container(auth=auth, db=db,
tracer=tracer)
```

Swap nursery for graduated: change one line in `compose.py`.

Failure attribution (Meta-Agent's 3 levels):

- **LOCAL**: this port's adapter is broken
- **UPSTREAM**: this port received bad data from the port it depends on
- **STRUCTURAL**: the composition in `compose.py` is wrong (wrong adapter wired, wrong order)

When something breaks, check in that order. Don't rewrite; localize.

How Ports Self-Improve (AEvo Pattern)

Each port has two pieces:

- `__init__.py` — the INTERFACE (protected; only humans change this)
- `nursery.py` / adapters — the IMPLEMENTATION (agents can strengthen these)

The rule: an agent working in any project can:

- Add a case the nursery didn't handle
- Strengthen validation in an adapter
- Add a new adapter for a capability the port didn't have yet

An agent may NOT:

- Change the interface in `__init__.py` (that's the protected evaluator)
- Remove a method from the interface
- Weaken a guarantee the interface makes

This is enforced by discipline and code review, not by a CI gate (yet). The `PROTECTED` comment in the interface file is the marker.

How Lessons Travel (FoT Pattern)

When a project uses a port and discovers something — a gotcha, a better pattern, a dead end — it writes to `substrate/insights/<port>.json`:

```
{
  "auth": [
    {
      "lesson": "Token expiry must be handled at the port boundary, not in the app",
      "source": "project-tenet",
      "date": "2026-06-08",
      "type": "gotcha"
    },
    {
      "lesson": "Never store peer-linkable
```

```
state in the auth record — use opaque
handles",
      "source": "project-sphinx-tahoe",
      "date": "2026-06-06",
      "type": "contract"
    }
  ]
}
```

When a new project starts, it copies `substrate/insights/` from the shared store.
The lessons are there before the first line of app code is written.

The distillation rule: when two lessons say the same thing, merge them.
Keep the insights file SHORT (~20 entries max per port). Quality over quantity.
A long insights file is a junk drawer. A short one is a cheat sheet.

Graduation: Nursery → Production

A port graduates when the nursery adapter stops being sufficient. Signs:

- The in-memory store fills up and you need persistence
- The local auth needs to handle OAuth from a real provider

- The stdout tracer isn't enough and you need searchable traces

Graduation is one change: in `compose.py`, swap the adapter.

```
# before
from substrate.ports.db.nursery import
SQLiteDB
db = SQLiteDB(path=":memory:")

# after graduation
from substrate.ports.db.postgres import
PostgresDB
db = PostgresDB(url=config.DATABASE_URL)
```

Nothing in `app/` changes. The port interface is the same. The data types crossing
the boundary are the same. Only the adapter behind the port changes.

Graduation is not a big bang. Graduate one port at a time. Start with the one
that's actually causing pain. Leave the rest as nursery until they hurt.

The Blocks From the Next.js Analysis, Mapped to Ports

From reading `ixartz/Next-js-Boilerplate`, these are the ports to build first:

Block	Port Name	Nursery Adapter	Graduated Adapter
HTTP Server	`transport`	stdlib http / minimal	production framework
Auth	`auth`	local hashed pw + token	Clerk / OAuth / LDAP
Database	`db`	SQLite in-memory	Postgres via pool
Schema	`schema`	baseline tables + migrations	domain tables + versioning
Env Validation	`config`	os.environ + fail-loud	Vault / secrets manager
Structured Logger	`tracing`	JSON to stdout	LangSmith / Datadog
Request Guard	`guard`	passthrough	Arcjet / rate-limiting
Input Validation	`validation`	Zod/Pydantic inline	schema registry

Block	Port Name	Nursery Adapter	Graduated Adapter
i18n	`i18n`	single locale	multi-locale + CDN
Error + Observability	`errors`	stderr + exit code	Sentry + alerting
Store (cross-run memory)	`store`	in-memory dict (LangGraph pattern)	sqlite / postgres
CI	`ci`	local test runner	full pipeline
Schema Migration	`migrations`	auto-generate + auto-run	versioned + rollback

---

What NOT to Build Yet

- The graduation trigger (when does nursery auto-upgrade) — undefined, leave it
- The FoT federation mechanism (auto-sync insights across repos) — do it manually first
- The agent that improves ports (AEvo loop) — write the ports first, then improve them

- Visual tooling, dashboards, CLIs — these are nice-to-have, not blockers

Start with the ports. Everything else is downstream of having real ports.

First Thing to Build

One working port, end to end, with:

1. `__init__.py` — the Protocol interface (20 lines)
2. `nursery.py` — the thinnest real implementation
3. A proof: an app that uses only the interface, with nursery swapped for a graduated adapter, with ZERO changes to the app

The auth port. Because it recurs in every project, the interface is simple (`get_user`, `protect`, `register`, `login`), and we already have a working proof of concept from the Python demo earlier today.

Build that. Ship it. Then add the next port. No more documents.